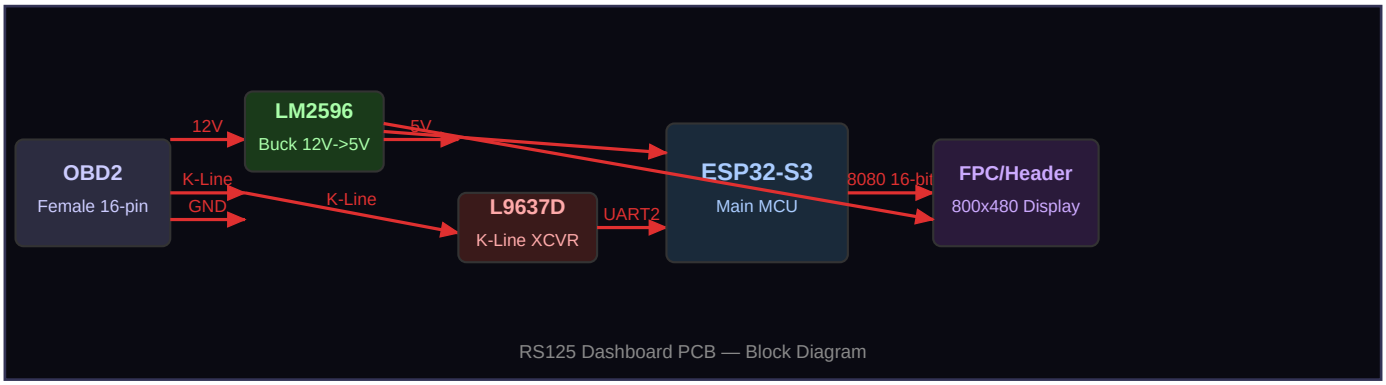
RS125 DASHBOARD PCB

OBD2 Interface + ESP32-S3 + 800×480 Display
Component List · Pin Connections · Design Choices

This document covers everything needed to design and build the custom PCB that sits between the Aprilia RS125's diagnostic port, the ESP32-S3 microcontroller, and the 800×480 parallel TFT display. It is intentionally concise — one page per topic.

1 — System Block Diagram



RS125 Dashboard PCB — Block Diagram

2 — Key Design Choices

Why ESP32-S3 and not original ESP32

The original ESP32 has no native parallel LCD controller. Driving an 800×480 display over 8080 16-bit parallel requires the ESP32-S3's built-in LCD peripheral, which supports up to 40 MHz pixel clock — giving ~52 fps full-frame. The original ESP32 would need SPI, limiting you to ~4 fps at this resolution. Not viable.

Why 8080 16-bit parallel and not SPI

At 800×480, a full frame is 768 KB. SPI at 27 MHz pushes that in ~228 ms (4 fps). 8080 16-bit parallel at 20 MHz pushes it in ~38 ms (26 fps), at 40 MHz in ~19 ms (52 fps). For a live RPM arc animating at 30+ fps, parallel is non-negotiable.

Why LM2596 for power

The bike supplies switched 12V (actually 13.6–14.4V when running). The ESP32-S3 needs 5V input (3.3V via onboard regulator), and the display needs 3.3V–5V depending on the specific panel. The LM2596 is a simple, proven, adjustable buck converter that handles up to 40V input, 3A output — enough for the ESP32-S3 (~500mA peak) plus display backlight (~300mA). Total draw under 1A comfortably.

Why L9637D for K-Line

The bike's ECU communicates over K-Line at 12V logic levels. The ESP32-S3 GPIO is strictly 3.3V tolerant — connecting 12V directly destroys the chip instantly. The L9637D is a single-chip automotive K-Line transceiver that handles the 12V↔3.3V conversion, bus pull-up, and ISO 9141 timing requirements. It also provides short-circuit and overvoltage protection on the bus line.

OBD2 female connector (board-mounted)

Rather than soldering wires to an OBD2 adapter cable, mount a standard 16-pin OBD2 female connector directly on the PCB. This gives a clean, vibration-resistant connection. The Aprilia 6-pin→OBD2 adapter cable plugs straight in. Only pins 4 (GND), 7 (K-Line), and 16 (12V) are wired on the PCB — the rest are NC (no connect).

Single-layer vs double-layer PCB

Go double-layer. The 16-bit data bus (DB0–DB15) plus control lines (WR, RS, CS, RST) is 20 signals between the ESP32-S3 and display connector. Routing these on a single layer without crossovers is nearly impossible at a reasonable board size. Double-layer at JLCPCB or PCBWay costs the same (~€5 for 5 boards) and makes routing trivial.

Display connector type

Most 800×480 parallel displays use a 40-pin or 50-pin 0.5mm pitch FPC (Flexible Printed Circuit) connector. Solder a matching ZIF (Zero Insertion Force) FPC socket onto the PCB. Verify the pin count and pitch of your specific display before ordering the connector — they vary. A safer alternative is a display module that already breaks out to a 2.54mm pitch header, which is much easier to hand-solder.

■ Keep the K-Line trace away from the 16-bit LCD data bus traces. K-Line runs at 12V transient levels and the L9637D switches at 10.4 kbps — crosstalk into the high-speed LCD lines can corrupt pixel data. Route K-Line on the opposite side of the board or with a ground pour between them.

3 — Component List (BOM)

Ref	Component	Part Number / Spec	Qty	Package	~Cost
U1	Microcontroller	ESP32-S3-WROOM-1 (N8R8)	1	Module	€4.50
U2	K-Line Transceiver	L9637D	1	SO-8	€1.80
U3	Buck Converter IC	LM2596S-5.0 (fixed 5V)	1	TO-263-5	€0.80
J1	OBD2 Connector	Female 16-pin OBD2 PCB mount	1	Through-hole	€2.50
J2	Display Connector	FPC 40-pin 0.5mm ZIF (verify!)	1	SMD ZIF	€0.60
J3	Debug Header	2.54mm 4-pin (TX/RX/3V3/GND)	1	Through-hole	€0.20
L1	Inductor	100µH, 3A, power (LM2596)	1	DO-201	€0.40
D1	Schottky Diode	1N5822 or SS34 (LM2596 catch)	1	DO-201/SMB	€0.25
C1	Input Cap (buck)	100µF 35V electrolytic	1	Radial 8mm	€0.20
C2	Output Cap (buck)	220µF 10V electrolytic	1	Radial 8mm	€0.20
C3	Decoupling U2 VCC	100nF ceramic X7R	1	0402/0603	€0.05
C4	Decoupling U2 VCC	100nF ceramic X7R	1	0402/0603	€0.05
C5	Decoupling ESP32 5V	10µF ceramic X5R	1	0805	€0.10
C6	Decoupling ESP32 5V	100nF ceramic X7R	1	0402/0603	€0.05
R1	K-Line pullup	510Ω 1/4W (L9637D LIN to 12V)	1	0402/0603	€0.05
R2	UART TX resistor	1kΩ (ESP32 TX to L9637D TXD)	1	0402/0603	€0.05
F1	Polyfuse	0.5A hold / 1A trip, 16V	1	1812 SMD	€0.30
LED1	Power LED	Red 0603 SMD + 1kΩ series R	1	0603	€0.05
SW1	Reset button	Tactile 4-pin SMD, 3x4mm	1	SMD	€0.10
PCB	2-layer PCB	~80×60mm, 1.6mm FR4	1	—	€5.00

■ The LM2596S-5.0 is the fixed 5V version — no adjustment potentiometer needed, simpler layout, one fewer component. If your display needs 3.3V directly, add a small AMS1117-3.3 LDO after the LM2596 (it only needs to drop 1.7V from 5V, very low dissipation).

Total estimated cost: ~€17 per board excluding display and bike-side adapter cable. JLCPCB can assemble most SMD components for an additional ~€8–12 (PCBA service).

4 — Pin Connection Tables

4.1 — OBD2 Connector (J1) → PCB

OBD2 Pin	Signal	Connects to	Note
4	Chassis GND	PCB GND plane	Primary ground
5	Signal GND	PCB GND plane	Tie to pin 4
7	K-Line	L9637D pin 5 (LIN)	ISO 9141-2 bus
16	+12V Battery	F1 → LM2596 Vin+	Switched ignition rail
1,2,3,6, 8–15	NC	—	No connect

4.2 — L9637D (U2) Pin Connections

L9637D Pin	Name	Connects to	Note
1	TXD	R2 (1kΩ) → ESP32-S3 GPIO16 (UART2 RX)	ECU data out → MCU in
2	GND	PCB GND	Power ground
3	VCC	LM2596 5V output	Supply
4	RXD	ESP32-S3 GPIO17 (UART2 TX)	MCU out → ECU
5	LIN	OBD2 pin 7 (K-Line)	Bus connection
6	INH	VCC via 10kΩ (optional)	Pull high to enable, or leave NC
7	WAKE	NC	Not used
8	VCC	LM2596 5V output	Supply (tie to pin 3)

4.3 — LM2596S-5.0 (U3) Pin Connections

LM2596 Pin	Name	Connects to	Note
1	Vin	F1 output (+12V from OBD2 pin 16)	12–14.4V input
2	Vout	L1 → output rail (+5V)	Switched output
3	GND	PCB GND plane	
4	Feedback	Output rail (fixed 5V version)	Internal, no external R needed
5	ON/OFF	GND (always on)	Tie low to keep enabled
—	L1	100µH between Vout pin and +5V rail	Switching inductor
—	D1	Cathode to +5V rail, anode to GND	Catch diode
—	C1	100µF across Vin to GND	Input decoupling
—	C2	220µF across Vout (+5V) to GND	Output filter

4.4 — ESP32-S3 → Display (8080 16-bit Parallel)

ESP32-S3 GPIO	LCD Signal	Description
GPIO0	DB0	Data bit 0 (LSB)

ESP32-S3 GPIO	LCD Signal	Description
GPIO1	DB1	Data bit 1
GPIO2	DB2	Data bit 2
GPIO3	DB3	Data bit 3
GPIO4	DB4	Data bit 4
GPIO5	DB5	Data bit 5
GPIO6	DB6	Data bit 6
GPIO7	DB7	Data bit 7
GPIO8	DB8	Data bit 8
GPIO9	DB9	Data bit 9
GPIO10	DB10	Data bit 10
GPIO11	DB11	Data bit 11
GPIO12	DB12	Data bit 12
GPIO13	DB13	Data bit 13
GPIO14	DB14	Data bit 14
GPIO15	DB15	Data bit 15 (MSB)
GPIO18	WR	Write strobe — clocks data into display on rising edge
GPIO19	RS / DC	Register select: LOW=command, HIGH=data
GPIO20	CS	Chip select — active LOW
GPIO21	RST	Hardware reset — active LOW, pull HIGH via 10kΩ
GPIO16	UART2 RX	K-Line data IN (from L9637D TXD via 1kΩ)
GPIO17	UART2 TX	K-Line data OUT (to L9637D RXD)
5V pin	VCC	5V from LM2596 to ESP32-S3 module VCC
GND	GND	Common ground

4.5 — ESP32-S3 Pin Diagram

5 — Firmware Integration Notes

Initialising the 8080 parallel bus in ESP-IDF / LVGL

ESP-IDF provides the esp_lcd driver which supports 8080 parallel natively on the ESP32-S3. The key configuration struct is:

```
esp_lcd_i80_bus_config_t bus_cfg = {
    .dc_gpio_num     = 19,   // RS pin
    .wr_gpio_num     = 18,   // WR pin
    .clk_src         = LCD_CLK_SRC_DEFAULT,
    .data_gpio_nums  = {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15},
    .bus_width       = 16,
    .max_transfer_bytes = 800*480*2,
};
esp_lcd_panel_io_handle_t io;
esp_lcd_panel_io_i80_config_t io_cfg = {
    .cs_gpio_num     = 20,
    .pclk_hz         = 20000000,  // 20 MHz — safe start
    .trans_queue_depth = 10,
    .lcd_cmd_bits    = 8,
    .lcd_param_bits  = 8,
};
```

Once the bus is initialised, plug in your display driver (ST7262, NT35510, or whichever controller your panel uses). LVGL then wraps this via lv_display_create() and lv_display_set_flush_cb().

K-Line UART configuration

```
uart_config_t uart_cfg = {
    .baud_rate  = 10400,
    .data_bits  = UART_DATA_8_BITS,
    .parity     = UART_PARITY_DISABLE,
    .stop_bits  = UART_STOP_BITS_1,
    .flow_ctrl  = UART_HW_FLOWCTRL_DISABLE,
};
uart_param_config(UART_NUM_2, &uart;_cfg);
uart_set_pin(UART_NUM_2, 17, 16, -1, -1);
```

■ Start the display at 10–15 MHz pixel clock during bring-up. Increase to 20–40 MHz only after confirming stable output. Some displays are picky about clock edge polarity — check your panel datasheet for PCLK active edge (rising vs falling).

Boot time

ESP-IDF boots in ~300ms to app_main(). LVGL init + display driver init adds ~100ms. The hex grid background (pre-rendered into a frame buffer at boot) takes ~500ms at 20 MHz. Total to first live frame: under 1 second. No fast-boot tricks needed.